StockFlow Case Study

Name : Abhishek Dani

This is my solution to the StockFlow backend case study.
I have tried to keep the implementation simple while clearly explaining the reasoning behind each decision.

Since the requirements were intentionally incomplete, I made a few assumptions upfront and focused on building a clean, correct, and explainable system.

Assumptions I Made

- SKUs are globally unique across the platform
- A product can exist without being assigned to a warehouse initially
- Low-stock alerts are generated per warehouse
- “Recent sales” means at least one sale in the last 30 days
- A product can have multiple suppliers, but one is returned
- If there are no recent sales, no alert should be generated

Part 1 : Code Review

Here are the key issues I found in the original API:

1. Two Separate Commits

- The product and inventory were committed separately.
- If inventory creation fails, the product still exists.
- This leads to inconsistent data.

Fix: Wrap both operations in a single transaction using flush() and commit once.

2. Product Tied to Single Warehouse

- The original design assumed one warehouse per product.
- This conflicts with requirement: products can exist in multiple warehouses

Fix: Use a separate Inventory table to map product ↔ warehouse.

3. No Input Validation

- The API directly accessed fields like data['name'].
- Missing fields would crash the API.

Fix: Validate required fields (name, sku, price) before processing.

4. SKU Uniqueness Not Enforced

- Duplicate SKUs could be inserted.
- Causes reporting and billing issues.

Fix: Check for existing SKU before insert.

5. Price Stored as Float

- Float causes precision errors.
- Example: financial calculations become inaccurate.

Fix: Use Decimal for price.

6. No Warehouse Validation

- Invalid warehouse IDs could be used.
- Leads to broken relationships.

Fix: Validate warehouse before creating inventory.

7. No Error Handling

- Failures would crash the API.
- Poor user experience.

Fix: Use try/except with rollback.

8. No Idempotency

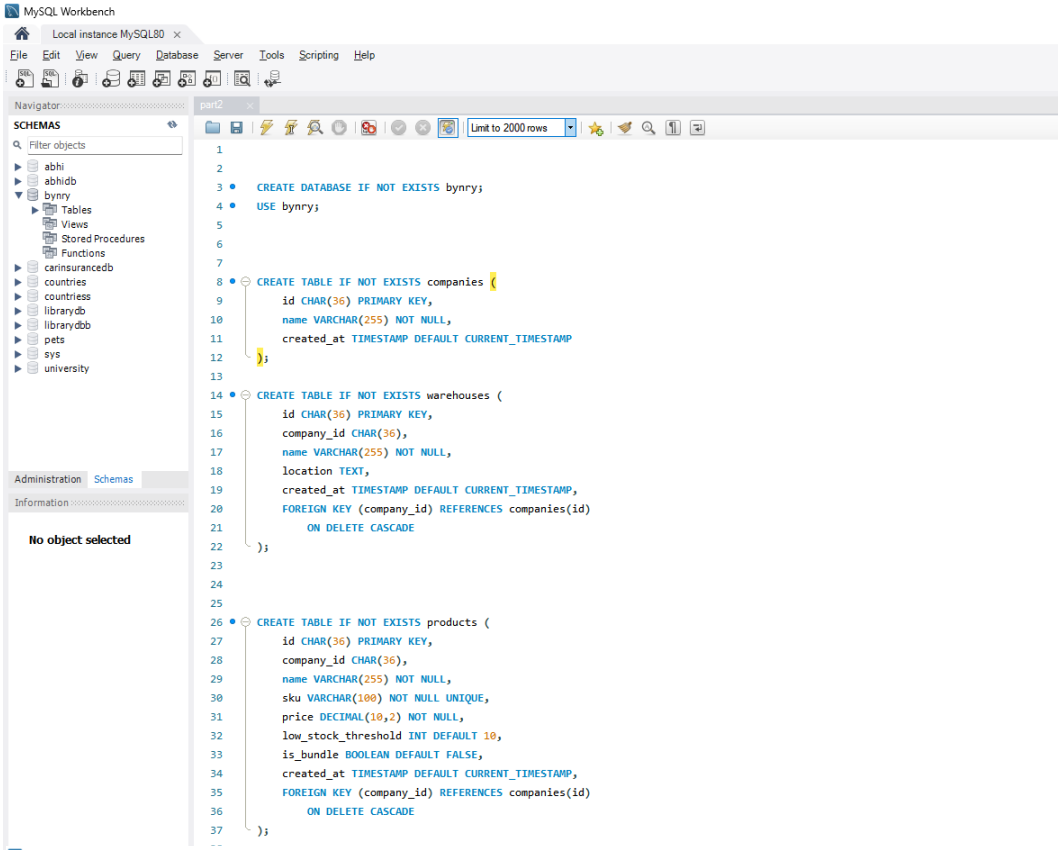
- Repeated requests could create duplicate data.
- Leads to inconsistent system behavior.

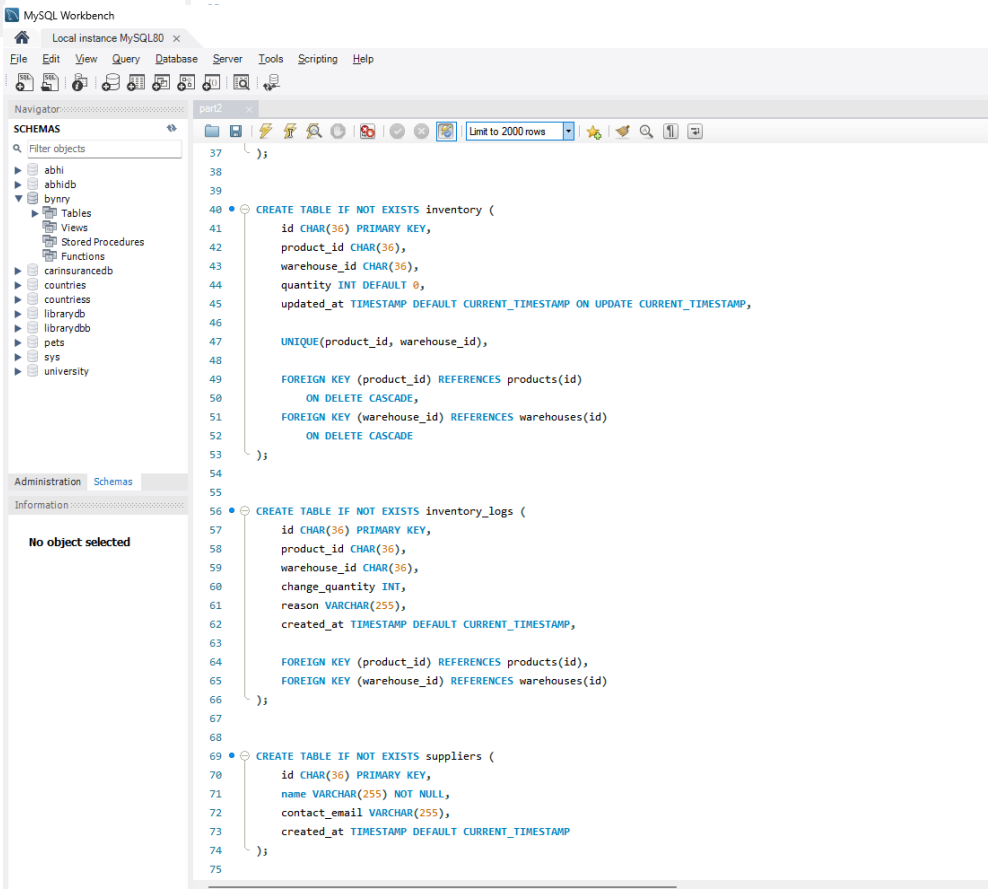
Summary

The main focus of fixes was:

- Data consistency
- Validation
- Correct data modeling
- Error handling

Part 2: Database Design





```sql

CREATE TABLE IF NOT EXISTS product_suppliers (
    product_id CHAR(36),
    supplier_id CHAR(36),

    PRIMARY KEY (product_id, supplier_id),

    FOREIGN KEY (product_id) REFERENCES products(id)
        ON DELETE CASCADE,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
        ON DELETE CASCADE
);


CREATE TABLE IF NOT EXISTS product_bundles (
    parent_product_id CHAR(36),
    child_product_id CHAR(36),
    quantity INT DEFAULT 1,

    PRIMARY KEY (parent_product_id, child_product_id),

    FOREIGN KEY (parent_product_id) REFERENCES products(id),
    FOREIGN KEY (child_product_id) REFERENCES products(id)
);


CREATE TABLE IF NOT EXISTS sales (
    id CHAR(36) PRIMARY KEY,
    product_id CHAR(36),
    quantity INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (product_id) REFERENCES products(id)
);


SHOW TABLES;
DESCRIBE products;
```

```sql
SHOW CREATE TABLE inventory;



INSERT IGNORE INTO companies (id, name)
VALUES ('c1', 'Demo Company');

INSERT IGNORE INTO warehouses (id, company_id, name)
VALUES ('w1', 'c1', 'Main Warehouse');

INSERT IGNORE INTO products (id, company_id, name, sku, price, low_stock_threshold)
VALUES ('p1', 'c1', 'Widget A', 'WID-001', 100.00, 20);

INSERT IGNORE INTO inventory (id, product_id, warehouse_id, quantity)
VALUES ('i1', 'p1', 'w1', 5);

INSERT IGNORE INTO suppliers (id, name, contact_email)
VALUES ('s1', 'Supplier Corp', 'orders@supplier.com');

INSERT IGNORE INTO product_suppliers (product_id, supplier_id)
VALUES ('p1', 's1');

INSERT IGNORE INTO sales (id, product_id, quantity, created_at)
VALUES ('sale1', 'p1', 10, NOW());


SELECT
    p.name,
    i.quantity,
    p.low_stock_threshold
FROM products p
JOIN inventory i ON p.id = i.product_id
WHERE i.quantity < p.low_stock_threshold;
```

Questions I would ask the product team:

1. Is low-stock threshold defined per product or per warehouse ?
2. What is the exact definition of “recent sales” (fixed 30 days or configurable) ?
3. Can a product have multiple suppliers, and how do we choose one (cheapest, preferred, fastest) ?
4. Can inventory go negative (for backorders) or must it always stay ≥ 0 ?
5. Should returns, transfers, or damages also update inventory or only sales ?
6. Is authentication required to restrict access to company-specific data ?
7. How should bundle products behave (reduce component stock automatically or not) ?

Design decisions and justification :

• Used a separate Inventory table to handle product–warehouse mapping because a product can exist in multiple warehouses.

• Used an InventoryMovement table to track stock changes over time, which helps in calculating sales rate and analyzing trends.

• Enforced unique SKU to prevent duplicate products and maintain consistency.

• Stored price as Decimal instead of float to avoid precision issues in financial calculations.

• Used a SupplierProduct table to support multiple suppliers per product and flexible sourcing.

• Added indexing on product_id, warehouse_id, and created_at to make stock lookup and recent sales queries faster.

• Followed normalization to avoid redundancy and keep the schema scalable and maintainable.

Part 3: Low-Stock Alerts API

Endpoint:

GET /api/companies/{company_id}/alerts/low-stock

The API returns products that are low on stock and require attention.

Logic used:

- For each product in the company, check its inventory across all warehouses
- Only consider entries where stock is less than or equal to the product's threshold
- From those, include only products that have at least one sale in the last 30 days
- Calculate average daily sales using total sales in last 30 days
- Estimate days until stockout using:
  current_stock / average_daily_sales
- Fetch supplier information for that product (if available)

Response includes:

- product_id, product_name, sku
- warehouse_id
- current_stock and threshold
- days_until_stockout
- supplier details (if exists, otherwise null)

Edge cases handled:

- If no recent sales → product is skipped (no alert)
- If average daily sales = 0 → days_until_stockout is set to null (avoid division error)
- If no supplier found → supplier field is null
- If product exists in multiple warehouses → each warehouse is evaluated separately
- If company has no products → empty alerts list returned

Reasoning:

- Only showing low stock is not enough; filtering by recent sales ensures alerts are actionable
- Separating inventory and movement data helps calculate sales trends efficiently
- Calculating days until stockout helps prioritize urgent items
- Keeping logic simple ensures readability and maintainability while still solving the business problem